

Resumen CDD - Ciencia de Datos: Guía de Estudio Completa

Sep 24, 2026 · @Alan Garber

Este documento resume las 8 clases de la materia (Ciencia de Datos, UTN FRBA) para preparar el examen: desde la introducción conceptual hasta Deep Learning, NLP y CNNs. Cada clase incluye definiciones, fórmulas con su interpretación, algoritmos paso a paso y el código de ejemplo usado en las diapositivas. Al final hay una tabla comparativa de técnicas de ML, un consolidado de métricas de evaluación, un glosario bilingüe y preguntas de examen probables.

Cómo usarlo: lee cada clase en orden la primera vez; para repasar antes del examen, andá directo a la sección de métricas, el glosario y las preguntas de examen.

Clase 1 — Presentación de Ciencia de Datos

¿Qué es la Ciencia de Datos?

La **Ciencia de Datos** es una disciplina interdisciplinaria que recolecta, procesa, analiza e interpreta grandes volúmenes de datos para extraer conocimiento útil que ayude a tomar decisiones. Combina:

- **Matemática y Estadística**
- **Informática y Programación**
- **Inteligencia Artificial**
- **Conocimiento del dominio** (negocios, salud, ciencia, etc.)

Su objetivo principal es transformar datos en información y conocimiento accionable.

Desde la **Ingeniería Informática**, la Ciencia de Datos se aborda como un problema de ingeniería de software, un problema algorítmico/computacional y un sistema escalable y eficiente. Se apoya en algoritmos y estructuras de datos, estadística computacional, sistemas distribuidos, e IA/ML.

Historia y evolución

Período	Hito
Siglo XVII-XIX	Desarrollo de la estadística (probabilidad, censos, estudios poblacionales); uso de datos para gobiernos y ciencia

Período	Hito
Siglo XX	Aparición de las computadoras; análisis estadístico automatizado; nacen "análisis de datos" y "minería de datos"
1980–1990	Auge de la minería de datos
Años 2000	Surgimiento del Big Data
Actualidad	Ciencia de Datos moderna: integra Big Data + Machine Learning + computación distribuida, arquitecturas escalables y automatización del análisis

Fundamentos previos clave: estadística computacional, bases de datos (modelo relacional de Codd → NoSQL), e ingeniería de software.

Las 5V del Big Data (años 2000):

1. **Volumen** — cantidad masiva de datos generados.
2. **Velocidad** — rapidez con la que se generan y deben procesarse.
3. **Variedad** — diversidad de formatos y fuentes (estructurados, no estructurados).
4. **Veracidad** — confiabilidad y calidad de los datos.
5. **Valor** — utilidad real que se extrae del dato procesado.

Importancia

La Ciencia de Datos permite: tomar mejores decisiones (basadas en datos reales, no solo intuición), detectar patrones y tendencias (comportamiento de usuarios, mercados, riesgos), optimizar recursos (costos, procesos, eficiencia) y anticipar el futuro (predicción de ventas, diagnósticos tempranos, prevención de fraudes).

Desde el rol del ingeniero informático: diseñar sistemas basados en datos, implementar algoritmos eficientes, integrar IA en sistemas reales y resolver problemas complejos mediante modelado computacional.

Ámbitos de aplicación

- **Salud:** diagnóstico de enfermedades, análisis de imágenes médicas, predicción de brotes epidemiológicos.
- **Economía y finanzas:** detección de fraudes, análisis de riesgos, predicción de mercados.
- **Empresas y marketing:** recomendaciones personalizadas, análisis del comportamiento del cliente, optimización de precios.

- **Transporte:** rutas eficientes, vehículos autónomos, predicción de tráfico.
- **Ciencia y medio ambiente:** cambio climático, análisis climático, conservación de especies.
- **Redes sociales y tecnología:** algoritmos de recomendación, moderación de contenidos, análisis de tendencias.
- **Específicas de Informática:** desarrollo de software inteligente (sistemas de recomendación), cloud computing y sistemas distribuidos, IA/ML (clasificación, regresión, Deep Learning, NLP), ciberseguridad basada en datos, IoT y edge computing.

Proceso de la Ciencia de Datos (pasos)

1. **Recolección de datos** — bases de datos, sensores, redes sociales, encuestas.
2. **Limpieza de datos** — eliminar errores, corregir datos incompletos, unificar formatos.
3. **Análisis de datos** — estadísticas, gráficos, modelos matemáticos.
4. **Modelado** — Machine Learning, IA, predicciones.
5. **Interpretación y comunicación** — informes, visualizaciones, toma de decisiones.

Metodología (visión de ingeniería)

- **Análisis del problema:** requerimientos funcionales/no funcionales, restricciones computacionales, arquitectura del sistema, diseño de pipelines (batch vs streaming).
- **Gestión de datos:** bases SQL y NoSQL, data lakes y data warehouses.
- **Preprocesamiento:** limpieza, transformación, feature engineering.
- **Modelado algorítmico:** selección de algoritmos, entrenamiento/validación, optimización (tuning).
- **Despliegue y mantenimiento:** APIs de modelos, CI/CD, monitoreo de desempeño.

Herramientas típicas

- **Lenguajes:** Python, Java, Scala, SQL.
- **Frameworks/librerías:** Scikit-learn, TensorFlow / PyTorch, Spark MLlib.
- **Infraestructura:** Hadoop, Apache Spark, Docker y Kubernetes, cloud (AWS, GCP, Azure).

Impacto social y ético

Desde la Ingeniería Informática, la ética en Ciencia de Datos implica: seguridad y privacidad de la información, diseño de sistemas transparentes y auditables, prevención de sesgos algorítmicos y cumplimiento normativo (protección de datos). El ingeniero informático es responsable del impacto técnico y social de los sistemas que diseña.

Idea central para el examen

La Ciencia de Datos, desde la Ingeniería Informática, se entiende como **ingeniería aplicada al conocimiento**: combina fundamentos teóricos con implementación práctica de sistemas inteligentes, escalables y eficientes.

Clase 2 — Introducción a Python

¿Por qué Python?

Legibilidad y simplicidad, open source (gran ecosistema y comunidad activa), cross-platform, y fuerte presencia en GUI, Ciencia de Datos, simulación numérica, desarrollo web y automatización de tareas.

Instalación:

- **Tradicional** (python.org): instala el lenguaje, el intérprete y `pip` (gestor de paquetes). Se verifica con `python --version`.
- **Anaconda** (recomendado en el curso): incluye Python + el gestor de entornos/paquetes `conda` + librerías preinstaladas.
- **Ventajas**: gestión de dependencias, manejo de múltiples versiones/entornos, interfaz gráfica (Navigator), instalación/actualización sencilla.
- **Desventajas**: mayor consumo de recursos, posibles conflictos `conda` vs `pip`, menos control sobre qué se instala.

Formas de ejecutar código: Notebooks (`.ipynb`, interactivos: código + texto + gráficos en un documento — Jupyter, Colab, VS Code) vs. Scripts (`.py`, se ejecutan de punta a punta — Spyder, Atom, VS Code).

Tipos de datos y operadores

Tipo	Nombre Python	Ejemplo
Entero	<code>int</code>	<code>3</code>
Real	<code>float</code>	<code>0.1</code>
Texto	<code>str</code>	<code>'texto'</code>
Booleano	<code>bool</code>	<code>True / False</code>

`type(objeto)` devuelve el tipo; `print()` imprime argumentos. Operadores numéricos: `+` `-` `*` `/` (la división siempre da `float`), `//` división entera, `**` potencia, `%` módulo (resto).

```
8 / 2      # 4.0 (float)
10 // 8    # 1  (entera)
10 % 8     # 2  (resto)
3 + 3 * 2  # 9  (precedencia de operadores)
```

Variables: nombre asignado a un valor en memoria (con `=`). Deben empezar con letra o `_`, sin espacios ni empezar con número; Python distingue mayúsculas/minúsculas. Se ejecuta línea por línea y se pueden sobrescribir.

f-strings: `print(f"2 + 3 es igual a {2+3}")` — forma más elegante de insertar variables en texto que concatenar con `+`.

Linters y PEP 8

Un **linter** detecta errores sintácticos/lógicos, violaciones de estilo, código ineficiente y variables sin usar. Populares: **Pylint** (más completo, evalúa calidad), **Flake8** (más liviano, sintaxis/estilo), **Pyflakes** (solo lógica).

PEP 8 es la guía de estilo oficial: 4 espacios de indentación, líneas no muy largas, espacios alrededor de operadores, nombres de variables claros en minúsculas separadas por `_`, comentarios claros sin abusar.

Booleanos y condicionales

Una variable `bool` solo puede ser `True` o `False`. Operadores de comparación: `>` `<` `>=` `<=` `!=` `==`; `in` verifica pertenencia a una secuencia. Se combinan con `and`, `or`, `not`.

Estructura completa `if / elif / else`:

```
if condicion_1:
    # instrucciones caso verdadero
elif condicion_2:
    # si el anterior fue falso
elif condicion_3:
    # ...
else:
    # si todos los casos anteriores fueron falsos
```

Las líneas de encabezado (`if`, `elif`, `else`, `for`, `while`, `def`, `class`) terminan en `:` y el bloque que las sigue va indentado 4 espacios.

Listas

Se definen con corchetes `[ ]`, elementos separados por comas; indexado desde 0 (y desde el final con índices negativos, `-1` = último). Pueden mezclar tipos.

Operación / método	Efecto
<code>lista + lista2</code>	Concatena en una lista nueva
<code>lista * n</code>	Repite los elementos <code>n</code> veces
<code>lista[a:b]</code>	Slicing (subconjunto)
<code>elemento in lista</code>	¿Está el elemento?
<code>len(lista)</code>	Cantidad de elementos
<code>.append(x)</code>	Agrega <code>x</code> al final
<code>.remove(x)</code>	Elimina la primera aparición de <code>x</code>
<code>.index(x)</code>	Índice de la primera aparición
<code>.count(x)</code>	Cuántas veces aparece <code>x</code>
<code>.sort()</code>	Ordena la lista

Ciclos: `for` y `while`

```

for n in range(len(verduras)):
    print(verduras[n])

for elemento in verduras:      # itera directo sobre los elementos
    print(elemento)

for indice, valor in enumerate(verduras):  # con índice
    print(f"Índice: {indice}, Valor: {valor}")

for nombre, edad in zip(nombres, edades):  # dos listas a la vez
    print(f"{nombre} tiene {edad} años")

indice = 0
while indice < len(verduras):  # while: no crea iterador propio
    print(verduras[indice])
    indice += 1

```

`for` recorre objetos de tamaño fijo (listas, diccionarios); `while` corre hasta que se cumpla/dejar de cumplirse una condición.

Tuplas, diccionarios y sets

- **Tuplas** (1, 2, 3) : como listas pero **inmutables**. Usos: datos constantes, claves de diccionario, retorno de múltiples valores. Se pueden desempaquetar: `a, b, c = tupla`.
- **Diccionarios** {clave: valor} : pares clave-valor, claves únicas e inmutables. Acceso con `dic["clave"]` o `.get("clave", default)`. Métodos: `.keys()`, `.values()`, `.items()`, `.pop()`, `.clear()`, `.update()`. Se itera con `for clave, valor in dic.items()`.
- **Sets** {1, 2, 3} : colección desordenada de elementos únicos, sin acceso por índice. Métodos: `.add()`, `.remove()` (error si falta), `.discard()` (no da error), `.union()`, `.intersection()`, `.difference()`, `.clear()`. Útiles para eliminar duplicados y operaciones de conjuntos.

Funciones

```
def saludar_persona(nombre="Nicolas"): # parámetro con valor por defecto
    print(f"Hola, {nombre}!")

def calcular_precio_total(productos: list[tuple[str, float]], impuesto: float = 0.21) -> float:
    """Calcula el precio total aplicando impuesto (type hints + docstring)."""
    total_sin_impuesto = sum([precio for _, precio in productos])
    return total_sin_impuesto * (1 + impuesto)
```

Variables globales vs. locales: un parámetro modificado dentro de una función no cambia la variable global (solo la copia local), salvo que se declare `global x`. Los objetos **mutables** (listas, dicts, sets) sí pueden modificarse dentro de una función sin `global` (p. ej. `lista.append(4)` afecta a la lista original); los **inmutables** (int, str, tuple) no.

Librerías

Se instalan con `pip` (`pip install nombre`, `==version` para versión específica, `--upgrade`, `pip uninstall`, `pip list`) y se importan con `import`:

```
import math
from math import sqrt
import math as m
from math import *
```

Preinstaladas: `math`, `datetime`, `re`, `random`, `os`, `sys`. Requieren instalación: `numpy`, `pandas`, `matplotlib`, `scikit-learn`.

Programación Orientada a Objetos (POO)

Paradigma que organiza el código en **objetos** (entidades con atributos y métodos).

Beneficios: reutilización de código, mejor organización, mayor legibilidad y mantenimiento.

Concepto	Definición
Clase	Molde/plantilla que define atributos y métodos de los objetos
Objeto	Instancia concreta creada a partir de una clase
Atributo	Característica/dato del objeto (variable dentro de la clase)
Método	Función que define un comportamiento del objeto

```
class Perro:
    def __init__(self, nombre, edad):  # constructor: se llama al crear el
objeto
        self.nombre = nombre          # atributo
        self.edad = edad               # atributo

    def ladrar(self):                  # método
        print(f"{self.nombre} está ladrando.")

    def __str__(self):                 # dunder method: representación en
texto
        return f"Nombre: {self.nombre}. Edad: {self.edad}"

mi_perro = Perro('Max', 6)
mi_perro.ladrar()  # Max está ladrando.
print(mi_perro)    # Nombre: Max. Edad: 6
```

`self` es siempre el primer parámetro de un método y representa la instancia. `__init__` es el **constructor** (inicializa atributos); `__str__` es un ejemplo de *dunder method* (método especial) que define cómo se imprime el objeto.

Clase 3 — Numpy, Pandas y Matplotlib

Pandas: estructuras básicas

Pandas se usa para manejo y análisis de datos estructurados (tablas). Soporta CSV, Excel, JSON, SQL. Instalación: `pip install pandas`.

- **Series:** estructura unidimensional (cualquier tipo de dato).
- **DataFrame:** estructura bidimensional (filas y columnas), cada columna puede tener un tipo distinto.

```
import pandas as pd

serie = pd.Series([10, 20, 30, 40])

data = {'Nombre': ['Juan', 'Ana', 'Luis'], 'Edad': [28, 22, 35], 'Ciudad':
['Buenos Aires', 'Rosario', 'Córdoba']}
df = pd.DataFrame(data)  # a partir de un diccionario de listas

# también: lista de listas + columns=[...], o lista de diccionarios (una fila
cada uno)
```

Lectura y persistencia de datos

```
df = pd.read_csv('archivo.csv', sep=';', encoding='utf-8')
df = pd.read_excel('archivo.xlsx', sheet_name='hoja_1', usecols=['col_1',
'Col_2'])

from sqlalchemy import create_engine
engine =
create_engine('postgresql+psycopg2://usuario:contraseña@localhost:5432/nombre
_db')
df.to_sql('nueva_tabla', con=engine, if_exists='replace', index=False)
query = "SELECT * FROM tabla"
df = pd.read_sql(query, con=engine)
```

Acceso, filtrado y limpieza

```
df['Nombre']          # columna -> Series (indexar una sola columna)
df[['Nombre', 'Edad']] # varias columnas -> DataFrame
df.iloc[0]           # fila por posición -> Series

df_filtrado = df[df['Edad'] > 30]
df_filtrado = df[df['Ciudad'].isin(['Córdoba', 'Mendoza'])]

df_sin_na = df.dropna(subset=['Ciudad'])
df.dropna(subset=['Ciudad'], inplace=True)  # inplace=True modifica sin
reasignar
```

Agregar filas/columnas:

```

df.loc[len(df)] = ['María', 30, 'Mendoza'] # agrega al
final por índice
nueva_fila = pd.DataFrame({'Nombre': 'María', 'Edad': 30, 'Ciudad':
'Mendoza'})
df = pd.concat([df, nueva_fila], ignore_index=True)

df['Nacionalidad'] = 'Argentina' # columna con valor fijo
df['Edad en 10 años'] = df['Edad'] + 10 # columna calculada entre
columnas
df['Clasificación'] = df['Edad'].apply(clasificar_edad) # columna a partir
de una función

```

Transformación de datos

Método	Ámbito	Qué hace
<code>apply(func)</code>	DataFrame/Series	Aplica una función a lo largo de un eje (filas o columnas)
<code>applymap(func)</code>	DataFrame	Aplica una función elemento a elemento en todo el DataFrame
<code>map(dic_o_func)</code>	Series	Mapea valores de una Serie usando un diccionario o función

Tablas pivot y joins

```

tabla_pivot = pd.pivot_table(df, values='Ventas', index='Año',
columns='Producto', aggfunc='sum')
tabla_pivot = pd.pivot_table(df, values='Ventas', index='Año',
columns='Producto', aggfunc=['sum', 'mean'])

# join al estilo SQL: requiere una columna pivote compartida
df_left = pd.merge(df, df_puntos, on='Nombre', how='left') # how:
left/right/inner/outer

```

Cuando no hay valor para una celda tras un `merge`, Pandas coloca `NaN` (valor nulo).

Tipos de atributos (features)

- **Categóricas/discretas** (cualitativas): **nominales/binarias** (sin orden: igualdad/desigualdad, ej. sexo) y **ordinales** (con orden: además mayor/menor, ej. calificación).

- **Numéricas** (cuantitativas, continuas o discretas): soportan suma, resta (a veces multiplicación/división).

Algunos atributos parecen numéricos pero son categóricos (ej. código postal = nominal, aunque sean números).

Operación	Nominal	Ordinal	Numérico
Distribución y frecuencia	Sí	Sí	Sí
Mediana, percentiles	No	Sí	Sí
Suma, resta	No	No	Sí
Media, desvío estándar	No	No	Sí
Ratios, % de variación	No	No	Sí

Limpieza y preparación de datos

Valores faltantes interfieren en el aprendizaje del algoritmo; muchos algoritmos no pueden operar con valores faltantes. Estrategias:

1. **Descartar mediciones** — quitar filas con el atributo faltante: `df.dropna(subset=["atributo"])`.
2. **Descartar atributo** — quitar la columna completa: `df.drop(columns=["atributo"])`.
3. **Definir un valor** — imputar con un valor por defecto (cero, media, mediana, moda): `df["atributo"].fillna(df["atributo"].median())`.
4. **Predecir un valor** — completar con un modelo entrenado sobre el subconjunto completo.

Duplicados: `df.drop_duplicates(inplace=True)`.

Codificación de atributos categóricos (los algoritmos prefieren numéricos):

- **Label Encoding**: cada categoría → un entero (0, 1, 2...). Riesgo: introduce un orden artificial entre categorías que no lo tenían.
- **One-Hot Encoding**: una columna binaria por categoría (1 si pertenece, 0 si no). Evita el orden artificial pero aumenta la dimensionalidad.

Feature Scaling (escalado de atributos)

Atributos en escalas muy distintas (ej. altura en metros vs. sueldo en miles) distorsionan algoritmos basados en distancias (como k-NN o SVM), porque la magnitud domina el cálculo de distancia euclidiana.

Normalización (Min-Max): lleva el atributo al rango fijo $[0, 1]$.

$$x_{\text{nuevo}} = (x - x_{\text{mín}}) \div (x_{\text{máx}} - x_{\text{mín}})$$

Donde x es el valor a normalizar, $x_{\text{mín}} / x_{\text{máx}}$ el mínimo/máximo del atributo. Útil aunque los datos no sean normales, pero **muy sensible a outliers** (porque usa el máx/mín).

Estandarización (Z-score): centra en la media con desvío estándar 1, rango variable.

$$x_{\text{nuevo}} = (x - \mu) \div \sigma$$

Donde μ es la media del atributo y σ su desvío estándar (la diapositiva original lo etiqueta como "varianza", pero la fórmula estándar de estandarización usa el desvío estándar — la varianza es σ^2). Menos sensible a outliers, pero rango variable (puede generar problemas en algunos algoritmos) y es menos interpretable si la distribución no es gaussiana.

Conviene encapsular estas transformaciones en funciones reutilizables: para aplicarlas repetidas veces, reutilizarlas en otros datasets/proyectos, migrarlas a producción y probar varias alternativas fácilmente.

Visualización de datos

Matplotlib (`import matplotlib.pyplot as plt`): librería base, gráficos de barras, histogramas, scatterplots, error-charts, usable en scripts, servidores web y notebooks.

Seaborn (`import seaborn as sns`): estilo mejorado por defecto, integración directa con Pandas, gráficos complejos más intuitivos.

Objetivo	Gráfico	Código típico
Distribución de una variable	Histograma	<code>plt.hist(data, bins=n)</code>
Distribución suavizada	KDE plot	<code>sns.kdeplot(data)</code>
Distribución + cuartiles/outliers	Boxplot	<code>sns.boxplot(x='categoría', y='valor', data=data)</code>
Distribución + densidad por categoría	Violin plot	<code>sns.violinplot(x='categoría', y='valor', data=data)</code>
Puntos individuales por categoría	Strip plot	<code>sns.stripplot(x='categoría', y='valor', data=data)</code>
Cantidad según categorías (barras)	Bar / stacked bar	<code>data.pivot_table(...).plot(kind='bar', stacked=True)</code>

Objetivo	Gráfico	Código típico
Correlación entre variables	Heatmap	<code>sns.heatmap(data.corr(), annot=True, cmap='coolwarm')</code>
Proporciones de un total	Pie / bar	<code>plt.pie(data['valor'], labels=data['categoría'], autopct='%1.1f%%')</code>
Relación entre dos variables numéricas	Scatter	<code>plt.scatter(x, y)</code> (tamaño/color pueden codificar una 3ª variable: <code>s=data['tamaño']</code>)

Interpretación clave: el histograma cuenta frecuencias por intervalos (bins); el KDE da una estimación suave de densidad sin depender del ancho del bin; el boxplot resume mediana, cuartiles y outliers; el heatmap de correlación permite detectar rápidamente qué atributos numéricos están relacionados entre sí.

Clase 4 — Introducción a Machine Learning y Clasificación binaria

Definición de aprendizaje (Tom Mitchell)

"Se dice que un programa de computadora aprende de la experiencia **E** con respecto a alguna tarea **T** y alguna medida de desempeño **P**, si su desempeño en T, medido por P, mejora con la experiencia E."

- **T (Tarea):** ej. clasificar emails como spam.
- **E (Experiencia):** el training set (emails ya clasificados).
- **P (Medida de desempeño):** ej. cuántos emails detectó correctamente del total.

No confundir el **modelo** (programa resultante) con el **algoritmo** (procedimiento que genera el modelo a partir de los datos).

ML vs. desarrollo tradicional de software

Desarrollo tradicional	Machine Learning
El programador escribe las reglas a mano, revisa errores manualmente y vuelve a programar	El modelo aprende las reglas a partir de datos (training set)
Programas más largos y rígidos, difíciles de mantener ante el cambio	Programas más cortos, fáciles de mantener, adaptables al cambio

Desarrollo tradicional	Machine Learning
Requiere identificar patrones manualmente	Generaliza los datos automáticamente buscando parámetros óptimos

ML es útil cuando: la lógica del programa se vuelve demasiado compleja/imposible de programar a mano, los patrones cambian en el tiempo, o hay que extraer conocimiento de datos complejos y masivos.

Clasificación de algoritmos de ML

Se pueden clasificar por 4 criterios no excluyentes:

1. **Tipo de tarea (T):** Clasificación, Regresión, Clusterización, Reducción de dimensiones, Asociación.
2. **Forma de generalizar** a nuevos datos.
3. **Aprendizaje incremental vs. batch.**
4. **Supervisión del aprendizaje:** Supervisado, No supervisado, Reforzado.

Por tipo de tarea:

- **Regresión:** predice un valor cuantitativo continuo (ej. precio de una casa, valor de una acción).
- **Clasificación:** divide en grupos/categorías predefinidas; el modelo se entrena con datos ya etiquetados y luego predice el grupo de nuevos elementos (ej. detectar transacciones fraudulentas, categorizar clientes riesgosos).

Por tipo de aprendizaje:

Tipo	Cómo funciona	Datos	Usos típicos	Algoritmos típicos
Supervisado	El data scientist guía al algoritmo con ejemplos etiquetados (muestra + resultado deseado)	Labeled data	Clasificación (clase discreta, ej. spam/no spam), Regresión (valor continuo, ej. precio)	k-NN, Regresión Lineal, SVM, Árboles de decisión, algunas redes neuronales

Tipo	Cómo funciona	Datos	Usos típicos	Algoritmos típicos
No supervisado	Identifica patrones sin ayuda humana; más difícil de evaluar/entender	Unlabeled data	Clustering (agrupar clientes por comportamiento), Reglas de asociación, Visualización y reducción de dimensiones	K-Means, PCA
Reforzado	El algoritmo aprende de recompensas y penalidades por las acciones que realiza, a lo largo de intentos/tiempo	Estados, acciones y función de recompensa $F(a, e) = \text{reward}$	Predecir la mejor acción a tomar	— (vehículos autónomos, juegos de estrategia como Go/Ajedrez, robótica)

La **reducción de dimensiones** (no supervisado) preserva características relevantes reduciendo la cantidad de dimensiones: entrenamientos más veloces, menor uso de espacio y, en general, mejores resultados (los algoritmos suelen estar limitados por la cantidad de dimensiones — "maldición de la dimensionalidad").

Principales problemas en ML

1. **Calidad y cantidad de datos** insuficiente.
2. **Ruido** en las mediciones.
3. **Outliers** (valores atípicos).
4. **Valores faltantes**.
5. **Falta de estandarización** entre features.
6. **Complejidad del modelo vs. tamaño del dataset**.
7. **Features irrelevantes**: menos features irrelevantes y más relevantes → mejor performance. Se resuelve con *feature engineering* (feature selection, feature extraction, creación de nuevas features).

Overfitting (sobreajuste): el algoritmo "aprende de memoria" el training set → mala generalización a datos nuevos. Más probable en modelos complejos. Hay que buscar el balance entre complejidad del modelo (+ regularización), los patrones reales vs. el ruido de los datos, y la cantidad de datos disponible.

Underfitting (subajuste): el modelo es demasiado simple para la naturaleza de los datos. Se corrige con un modelo más poderoso, más feature engineering, o reduciendo las restricciones del modelo.

Clasificación binaria

Problema donde las observaciones se agrupan en una de **dos clases posibles**; cada predicción es una decisión entre dos opciones. Ejemplos: spam/no spam, enfermo/no enfermo, transacción legítima/fraudulenta, perro/gato, noticia real/fake news, llueve/no llueve.

Pipeline típico:

1. **Recolección de datos** con features relevantes (edad, ingresos, etc.).
2. **Preprocesamiento:** limpieza (eliminar datos faltantes) y codificación de variables categóricas (ej. masculino/femenino → 0/1).
3. **División del dataset:** entrenamiento y prueba (ej. 80%-20%).
4. **Entrenamiento del modelo** con un algoritmo de clasificación binaria (ej. árbol de decisión).
5. **Evaluación:** medir el rendimiento con métricas (ej. precision o recall — ver Clase 5).

Árbol de decisión

Modelo predictivo simple y poderoso que representa decisiones y sus consecuencias en forma de árbol. Divide el dataset en subconjuntos más pequeños según características y valores de los datos. Se usa en clasificación y regresión.

Cómo se arma (paso a paso conceptual): en cada nodo se elige el atributo (y umbral, si es continuo) que mejor separa las clases; se crea una rama por cada valor/condición posible de ese atributo, y se repite recursivamente en cada subconjunto resultante hasta llegar a las hojas (que asignan una etiqueta de clase). Ejemplo del curso: para decidir si "Juan sale a caminar", el árbol primero pregunta por el **Cielo** (Sol/Lluvia/Nublado); bajo "Sol" pregunta por la **Humedad** (Alta/Normal); bajo "Lluvia" pregunta por el **Viento** (Fuerte/Débil).

Frontera de decisión: línea o superficie que separa las clases en el espacio de características. En un árbol de decisión binario para atributos continuos, la frontera tiene forma **jerárquica** (cada región se divide en exactamente 2 regiones, o ninguna) y **rectangular**; cada hoja representa una región del espacio a la que se le asigna una etiqueta de clase.

Métrica básica: Accuracy

A(modelo) = Predicciones correctas ÷ Cantidad de predicciones

Proporción de predicciones correctas sobre el total. Es muy sencilla pero tiene varios problemas (se amplía en la Clase 5, ej. "accuracy paradox").

División de datos: train / test / validation

- **Train/Test Split:** separar una parte de los datos (Test Set) para evaluar el modelo antes de producción. Estrategia **random**: tomar, ej., 20% de las observaciones al azar.
- Comparar **Training Error** (sobre el training set) vs. **Generalization Error** (sobre el test set): si el error de entrenamiento es bajo pero el de generalización es alto → **overfitting**.
- **Validation Set:** se entrena con el training set, se optimiza y selecciona el mejor modelo según el validation set, y recién al final se chequea con el test set (una sola vez, cuando ya se está seguro del mejor modelo). División típica: 60% train / 20% validation / 20% test.
- **Stratified sampling:** el dataset se divide en grupos homogéneos ("stratas") mutuamente excluyentes; de cada strata se selecciona una cantidad representativa de observaciones (aplicando random sampling dentro de cada strata) — evita que el test set quede desbalanceado respecto a las clases.

Cross-Validation (k-fold)

1. Dividir el training set en **k folds**.
2. Para cada iteración (k iteraciones en total): entrenar el modelo con los **k-1** folds restantes y validar con el fold que quedó afuera.
3. El modelo final se entrena con el training set completo (una vez elegidos los hiperparámetros óptimos), y el error de generalización se mide con el test set.

Optimización de hiperparámetros

Un **hiperparámetro** es un valor configurado *antes* del entrenamiento (no lo aprende el modelo) que influye en su performance (ej. **max_depth** de un árbol).

- **Grid Search:** prueba todas las combinaciones posibles a partir de una lista de valores.
- **Randomized Search:** prueba N combinaciones de valores al azar (más eficiente cuando el espacio de búsqueda es grande).

Clasificadores multiclase

Cuando hay más de 2 clases:

- Con **clasificadores multiclase nativos** (ej. Random Forest).
- Con **clasificadores binarios** combinados mediante una estrategia (ej. SVM).

Estrategia	Cómo funciona	Cantidad de clasificadores	Predicción	Ventaja / Desventaja
OvA (One-vs-All / One-vs-Rest)	Un clasificador por clase (ej. Triángulos vs. No-Triángulos)	n clasificadores para n clases	La clase con mayor score entre todos los clasificadores	Simple, pero cada clasificador ve datos desbalanceados
OvO (One-vs-One)	Un clasificador por cada par de clases	$n \cdot (n-1) / 2$ clasificadores	La clase que gana en más pares (votación)	Ventaja: cada uno se entrena en un subconjunto pequeño (2 clases). Desventaja: mayor costo computacional (muchos clasificadores)
Multi-label	Generalización: cada instancia puede tener más de un target/etiqueta binaria simultáneamente (ej. detectar múltiples animales en una foto)	—	Un conjunto de etiquetas por instancia (no una sola clase)	Útil cuando las categorías no son mutuamente excluyentes

Caso práctico: Titanic (clasificación binaria con árbol de decisión)

Dataset: ~891 instancias, 12 características, clase binaria (sobrevivió=1 / no sobrevivió=0).

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Cargar y limpiar
data = pd.read_csv('titanic.csv')
data = data[['Age', 'Fare', 'Pclass', 'Sex', 'Survived']].dropna()
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1}) # codificar
variable categórica
```

```
# X e y, split train/test
X = data[['Age', 'Fare', 'Pclass', 'Sex']]
y = data['Survived']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Entrenar
model = DecisionTreeClassifier(max_depth=3) # max_depth: hiperparámetro
model.fit(X_train, y_train)

# Evaluar
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy) # Accuracy: 0.8212290502793296
```

`max_depth` limita la altura máxima del árbol (a mayor profundidad, más riesgo de overfitting). Se puede visualizar el árbol resultante para ver cómo clasifica según cada atributo.

Clase 5 — Evaluación de modelos de clasificación y regresión

Accuracy y su paradoja

$$A(\text{modelo}) = (TN + TP) \div (TN + TP + FN + FP)$$

Accuracy Paradox: con clases desbalanceadas, un modelo con **mayor** accuracy puede ser **peor** (menos útil) que uno con menor accuracy. Ejemplo del curso: si solo el 10% de las imágenes son "5", un modelo que **siempre** predice "no es 5" ya tiene 90% de accuracy sin haber aprendido nada. En el ejemplo numérico de la clase, un "Clasificador Fijo" que nunca predice positivo (0 TP, 0 FP) obtiene 98.5% de accuracy, superando al Clasificador A (98%) que sí detecta positivos — pero es completamente inútil para detectar la clase de interés.

Matriz de confusión (Confusion Matrix)

	Predicción: No	Predicción: Sí
Real: No	TN (True Negative)	FP (False Positive) — Error tipo I
Real: Sí	FN (False Negative) — Error tipo II	TP (True Positive)

- **TP (True Positive):** predijo positivo y era positivo.
- **TN (True Negative):** predijo negativo y era negativo.

- **FP (False Positive / Error tipo I):** predijo positivo pero era negativo ("falsa alarma").
- **FN (False Negative / Error tipo II):** predijo negativo pero era positivo ("lo dejó pasar").

Precision y Recall

$$P(\text{modelo}) = TP \div (TP + FP) \cdot R(\text{modelo}) = TP \div (TP + FN)$$

- **Precision:** de todo lo que el modelo predijo como positivo, ¿qué fracción era realmente positiva? (exactitud de las predicciones positivas — penaliza los falsos positivos).
- **Recall (Sensitivity):** de todo lo que realmente era positivo, ¿qué fracción detectó el modelo? (exactitud sobre las instancias reales positivas — penaliza los falsos negativos).

Ejemplo del curso (naranjas vs. manzanas): de 10 elementos predichos como "naranja", 7 eran naranjas realmente → Precision = 7/10. De 9 naranjas reales, el modelo detectó 7 → Recall = 7/9.

Decision Threshold (umbral de decisión)

Muchos clasificadores no devuelven directamente una clase sino un **score** (ej. una "probabilidad"); se predice la clase positiva si `score > umbral T`. Al **mover el threshold** se genera un trade-off:

- Subir el umbral → el modelo es más exigente para predecir positivo → sube la Precision, baja el Recall.
- Bajar el umbral → el modelo predice positivo más fácilmente → sube el Recall, baja la Precision.

Selección del threshold según el contexto:

- Clasificar publicidad "apta para todo público": conviene priorizar **Precision** (evitar marcar como mainstream algo que no lo es).
- Detectar fraude en tarjetas de crédito: conviene priorizar **Recall** (no dejar pasar ningún fraude, aunque haya más falsas alarmas).

F1-Score

Combina Precision y Recall en una sola métrica mediante la **media armónica** (más sensible a valores bajos que el promedio simple — penaliza fuerte si uno de los dos es bajo):

$$F1(\text{modelo}) = 2 \times [P(\text{modelo}) \times R(\text{modelo})] \div [P(\text{modelo}) + R(\text{modelo})]$$

Favorece a los clasificadores con Precision y Recall similares y altos; es la métrica útil cuando se necesita un balance entre ambas (y no hay una razón clara para priorizar una sobre la otra).

Curva ROC y AUC

ROC = *Receiver Operating Characteristic* (originada en detección de señales). Se construye comparando:

$$\text{TPR} = \text{TP} \div (\text{TP} + \text{FN}) \cdot \text{FPR} = \text{FP} \div (\text{FP} + \text{TN})$$

- **TPR (True Positive Rate)**: de las instancias realmente positivas, cuántas se predicen como positivas (= Recall).
- **FPR (False Positive Rate)**: de las instancias realmente negativas, cuántas se predicen (erróneamente) como positivas.

La curva ROC se obtiene variando el threshold, calculando (TPR, FPR) para cada valor y graficándolo. El punto (0,1) es la clasificación perfecta; la diagonal es la "línea de no discriminación" (equivalente a un clasificador aleatorio).

AUC (Área bajo la curva): valor entre 0.5 y 1 que resume el rendimiento del modelo separando las clases. Más cerca de 1 = mejor desempeño; 0.5 = equivalente a azar. Se usa para comparar clasificadores entre sí (mayor AUC = mejor, en general).

Curvas de aprendizaje y de validación

- **Curva de aprendizaje (learning curve)**: relaciona el **tamaño del dataset de entrenamiento** con el accuracy (u otra métrica), medida tanto en training como en un mismo set de validación fijo. Sirve para determinar si conviene agregar más datos.
- **Curva de validación / complejidad**: relaciona el valor de un **hiperparámetro** con el accuracy (u otra métrica) en training y validación. Sirve para identificar en qué regiones del hiperparámetro hay overfitting (training bueno, validación mal) o underfitting (ambos malos).

Métricas para regresión

La **regresión lineal** modela $y \approx f(X) + \text{error}$, donde $f(X)$ es una función lineal de $N+1$ parámetros. Aumentar el grado (regresión polinomial) aumenta la complejidad (riesgo de overfitting) pero permite capturar relaciones más complejas (evita underfitting).

$$\text{RMSE} = \sqrt{\frac{1}{m} \times \sum (h(x_i) - y_i)^2}, \text{ sumando para } i = 1 \text{ hasta } m$$

$$\text{MAE} = \frac{1}{m} \times \sum |h(x_i) - y_i|, \text{ sumando para } i = 1 \text{ hasta } m$$

Donde $x(i)$ es el vector de features de la medición i , $y(i)$ el valor real (label), $h(x(i))$ la predicción del modelo, y m la cantidad de mediciones.

- **RMSE (Root Mean Square Error)**: mide el desvío estándar del error de las predicciones; al elevar al cuadrado, **penaliza mucho más los errores grandes**. Muy usado en regresión.

- **MAE (Mean Absolute Error):** mide la distancia absoluta promedio entre predicción y valor real; **menos sensible a outliers/valores extremos** que RMSE porque no eleva al cuadrado.

Cuándo usar cada métrica (resumen práctico)

Situación	Métrica recomendada
Clases balanceadas, sin preferencia entre errores	Accuracy
Clases desbalanceadas	Precision, Recall, F1, AUC (no Accuracy)
Falsos positivos son costosos (ej. marcar contenido legítimo como spam)	Precision
Falsos negativos son costosos (ej. no detectar fraude o enfermedad)	Recall
Se necesita un balance entre Precision y Recall	F1-score
Comparar clasificadores independientemente del threshold elegido	Curva ROC / AUC
Regresión, penalizar fuerte los errores grandes	RMSE
Regresión, robustez ante outliers	MAE

Clase 6 — SVM, Ensamblés e Introducción a Deep Learning

Bias vs. Variance

El **error de generalización** (lo que impide que el modelo generalice bien a instancias nuevas) se descompone en:

$$\text{error}(X) = \text{bias}(X)^2 + \text{variance}(X) + \text{variance}(\epsilon)$$

- **Bias Error (sesgo):** suposiciones incorrectas del modelo (sesgo inductivo) — relacionado con el modelo elegido, no con los datos.
- **Variance Error (varianza):** sensibilidad a pequeñas fluctuaciones del training set — relacionado con los datos.
- **Irreducible Error / Ruido (ϵ):** error propio de los datos (ruido inherente a la medición o al fenómeno) que ningún modelo puede eliminar, por más complejo que sea — no depende ni del modelo ni de cómo se entrena.

Analogía del tiro al blanco: las predicciones son puntos, el target es el centro. **Bias** = distancia promedio de los puntos al centro; **Variance** = qué tan dispersos están los puntos entre sí. Existe un **trade-off**: a mayor complejidad del modelo, típicamente baja el bias pero sube la variance (y viceversa); se busca el punto de complejidad óptima que minimiza el error total.

Support Vector Machines (SVM)

Muy usados (pre-2010) para datasets complejos de tamaño chico/mediano. La idea es encontrar la "**calle**" (**margen**) **más ancha posible** entre las instancias de dos clases, delimitada por los **vectores de soporte** (las instancias ubicadas justo en los bordes de la calle). Nuevos datos que caen fuera de la calle no modifican la frontera de decisión ya encontrada.

Hard Margin: ninguna instancia puede estar dentro de la calle (no permite violaciones de margen).

- Limitación 1: exige que el dataset sea estrictamente separable linealmente → **bias error**.
- Limitación 2: muy sensible a outliers → **variance error**.

Soft Margin: permite un cierto número de errores (violaciones de margen), controlado por un hiperparámetro de **intolerancia** (`C` en scikit-learn):

- Más intolerancia → menos violaciones, calle más delgada (más riesgo de overfitting).
- Menos intolerancia → más violaciones, calle más ancha (mejor generalización, menos overfitting).

Kernel trick (clasificación no lineal): cuando los datos no son separables linealmente en el espacio actual, se los proyecta a un espacio de **mayor dimensión** (agregando M dimensiones adicionales) donde sí exista un hiperplano separador. Un **kernel** es la función de mapeo entre espacios. Cuidado: muchas features agregadas implican mayor costo computacional.

Kernel	Cuándo usarlo
Linear	Regla general: probar primero — más rápido y sencillo
Gaussian RBF	Si el dataset no es demasiado extenso
Polynomial, Sigmoid	Alternativas según la forma de los datos

Si hay tiempo y poder de cómputo disponible, conviene combinar la elección del kernel con **Grid Search + Cross Validation**.

Ensamblados (Ensemble Methods)

Idea general: crear **múltiples modelos** y combinarlos para producir mejores resultados que un modelo individual.

Familia	Mecanismo	Mejora principalmente	Ejemplos
Averaging methods	Modelos independientes, se promedian/votan sus predicciones	Variance error	Bagging, Random Forest
Boosting methods	Entrenamiento secuencial: cada modelo potencia las debilidades del anterior	Bias error	AdaBoost

Voting Classifiers: se entrenan múltiples modelos (algoritmos distintos) sobre el mismo training set; cada uno predice ("vota") sobre la misma instancia y los votos se agregan en una única predicción.

- **Hard Voting:** la predicción final es la clase más votada (moda): $\text{predicción} = \text{moda}\{C_1(X), C_2(X), \dots, C_n(X)\}$.
- **Soft Voting:** se pondera la probabilidad de cada clasificador por un peso w , se promedia por clase y gana la de mayor promedio ponderado.

Bagging (Bootstrap Aggregating): reduce el *variance error*. Se crean múltiples modelos con el **mismo algoritmo**, cada uno entrenado con un subdataset generado por **bootstrap sampling** (muestreo aleatorio **con reemplazo**, de tamaño m igual al original — puede haber muestras duplicadas y otras ausentes). Predicción final: moda (clasificación) o promedio (regresión) de todos los modelos.

- *Bagging* = copia la muestra (con reemplazo); *Pasting* = corta la muestra (sin reemplazo).
- **Out-of-bag evaluation:** en cada modelo, ~63% de las instancias del dataset original quedan incluidas en su subset (a veces repetidas) y ~37% quedan **fuera** ("out-of-bag"). Estas instancias no usadas varían de modelo a modelo y se pueden usar como un validation set gratuito, promediando el error entre modelos.
- **Random Forest** = Bagging aplicado específicamente a Árboles de Decisión. Comparado con un árbol individual: frontera de decisión menos compleja, menor variance, bias similar, menor overfitting y mejor generalización.

Boosting (AdaBoost — Adaptive Boosting): convierte modelos débiles (apenas mejores que el azar) en un modelo fuerte, reduciendo el *bias error*. Entrenamiento **secuencial** con el mismo dataset y algoritmo:

1. Empezar con un modelo débil entrenado sobre el dataset.

2. Las instancias mal clasificadas incrementan su **peso**; el modelo se pondera según su error general sobre el dataset.
3. Se entrena un segundo modelo, dando **mayor peso** a las instancias que el anterior clasificó mal.
4. Se repite: nuevas instancias mal clasificadas vuelven a ajustar los pesos, y cada modelo se pondera por su error.
5. Al llegar a **n** modelos (o a un modelo perfecto), la predicción final es una **votación ponderada por el peso de cada modelo**.

Introducción a Deep Learning

Deep Learning es una rama del Machine Learning que usa redes neuronales con **múltiples capas** ("profundas"); el modelo aprende a representar las features por sí mismo, sin necesidad de que un humano las diseñe a mano (a diferencia del ML tradicional). Permite procesar grandes volúmenes de datos y automatizar tareas complejas (voz, imágenes, texto).

	Machine Learning "clásico"	Deep Learning
Extracción de features	Manual (feature engineering por el ingeniero)	Automática, aprendida por la red
Modelos típicos	Regresión lineal, árboles, SVM	Redes neuronales profundas
Datos no estructurados (imagen, audio, texto)	Escala mal	Funciona bien, es su fortaleza
Ventajas	Más interpretable, requiere menos datos/cómputo	Escalable con más datos, versátil, sin feature engineering manual
Desventajas	Menor capacidad de patrones complejos	Necesita muchos datos y poder de cómputo (GPU), menos interpretable ("caja negra")

Componentes de una red neuronal

Ingredientes: capa de entrada, capa de salida, N capas ocultas (*fully connected*), **pesos** y **bias**, **funciones de activación** y **función de pérdida**.

- **Pesos (W)**: cada conexión entre neuronas tiene un peso que indica cuánta influencia ejerce una neurona sobre otra; se actualizan durante el entrenamiento.

- **Bias (b):** cada neurona tiene un bias que le permite ajustar su salida independientemente de las entradas, dando más flexibilidad al modelo.
- Los pesos se representan como **matrices** y los estados de una capa como **vectores** — esto es clave para entrenar redes rápidamente usando GPUs.

Funciones de activación (transforman la salida de una neurona antes de pasarla a la siguiente capa; introducen **no linealidad**, sin la cual la red sería solo una combinación lineal de las entradas, limitada a problemas lineales sin importar cuántas capas tenga):

Función	Qué hace	Dónde se usa típicamente
ReLU (Rectified Linear Unit)	Convierte negativos en 0, deja los positivos igual	Capas ocultas de la mayoría de redes profundas (facilita el entrenamiento)
Sigmoid	Convierte la salida en un valor entre 0 y 1	Última capa, clasificación binaria (da una "probabilidad")
Softmax	Generaliza Sigmoid: produce una distribución de probabilidad sobre varias clases (suma 1)	Última capa, clasificación multiclase

Funciones de pérdida (Loss functions): miden la discrepancia entre la predicción de la red y el valor real; guían el entrenamiento.

Tipo de problema	Función de pérdida	Cuándo usarla
Regresión	MSE	Penalizar fuerte los errores grandes
Regresión	MAE	Penalizar uniformemente, sin que outliers dominen
Regresión	Huber	Equilibrio entre MSE y MAE
Clasificación binaria	Binary Cross-Entropy	Estándar (ej. detección de fraude, diagnóstico médico)
Clasificación multiclase (excluyente)	Categorical Cross-Entropy	Cada muestra pertenece a una sola clase
Clasificación multiclase (muchas clases)	Sparse Categorical Cross-Entropy	Gran cantidad de categorías (ej. reconocimiento de imágenes)

Descenso de gradiente y Backpropagation

Algoritmo más usado para optimizar los pesos de la red (probar valores manualmente sería ineficiente):

1. **Extracción de datos:** tomar un lote de muestras X con sus objetivos y_{true} .
2. **Predicción (forward pass):** calcular y_{pred} pasando X por la red.
3. **Cálculo de pérdida (loss):** medir qué tan lejos está y_{pred} de y_{true} .
4. **Ajuste de pesos (backpropagation + descenso de gradiente):** mover todos los pesos en la dirección que minimiza la pérdida, de forma simultánea y eficiente (requiere derivadas parciales).

Learning Rate: hiperparámetro clave que controla el tamaño de los pasos del optimizador.

- Tasa **alta**: converge más rápido, pero riesgo de saltarse el mínimo o volverse inestable.
- Tasa **baja**: converge de forma más precisa, pero puede ser muy lento o quedar atrapado en mínimos locales.

Problemas del descenso de gradiente "puro" (batch): computacionalmente costoso con datasets grandes (cada iteración evalúa todos los datos), y puede quedar atrapado en mínimos locales sin llegar al mínimo global.

Descenso de gradiente estocástico (SGD) / Mini-batch: en lugar de usar todo el dataset, usa un único dato o un pequeño lote (**mini-batch**) aleatorio por iteración.

- Más rápido por iteración, pero el gradiente calculado es más "ruidoso" (menos preciso).
- Puede escapar de mínimos locales y zonas planas más fácilmente gracias a su naturaleza estocástica.
- Es el estándar en la práctica por su eficiencia computacional.
- **1 época** = un ajuste sobre todo el dataset; con mini-batches hay varias actualizaciones de pesos por época.

TensorFlow / Keras (flujo típico)

```
from tensorflow import keras
from tensorflow.keras import layers

# 1. Arquitectura: lista secuencial de capas densas (fully connected)
model = keras.Sequential([
    layers.Dense(512, activation="relu"),
    layers.Dense(10, activation="softmax")    # 10 clases -> softmax
])

# 2. Compilación: optimizador, función de pérdida y métricas
model.compile(optimizer="rmsprop",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])

# 3. Entrenamiento (fit): ajusta el modelo a los datos de entrenamiento
model.fit(train_images, train_labels, epochs=5, batch_size=128)

# 4. Evaluación en test
test_loss, test_acc = model.evaluate(test_images, test_labels)
```

Ejemplo del curso: dataset **MNIST** (imágenes de dígitos escritos a mano, 0-9, codificadas como arrays de NumPy). Con este modelo se obtuvo ~98.9% de accuracy en entrenamiento y ~97.8% en test — la brecha entre ambos es un ejemplo de **overfitting**. Un learning rate muy bajo demora la reducción de la pérdida; uno muy alto puede impedir que el modelo aprenda.

Otros frameworks: **PyTorch** y **JAX** (más usados en investigación, dan mayor control interno pero requieren más conocimiento; JAX está optimizado para hardware específico). Para otros tipos de input (imágenes más complejas, audio, video, texto) se usan capas especiales: **redes convolucionales** (Clase 8), **mecanismos de atención** (base de los LLMs) y **redes recurrentes**.

Clase 7 — Natural Language Processing (NLP)

¿Qué es el NLP?

Conjunto de modelos y técnicas capaces de entender el lenguaje humano. Enfoque **tradicional**: para cada problema específico existían algoritmos especializados, basados en reglas lingüísticas o en datos (con cientos de miles de modelos distintos). Técnicas principales para procesar texto: **Bag of Words**, **TF-IDF** y **Embeddings**.

Preprocesamiento de texto

Tokenización: proceso de convertir un texto en *tokens* (unidades mínimas) que serán el input del modelo. Suele ir acompañado de remoción de stopwords, stemming, remoción de puntuación, n-gramas, etc.

Tipo de tokenización	Cómo funciona	Ventaja	Desventaja
Por palabras	Separa por espacios/puntuación	Muy intuitivo para humanos	El vocabulario se vuelve gigante
Por caracteres	Cada letra es un token	Vocabulario minúsculo, nunca falla con palabras nuevas	Secuencias muy largas, se pierde el significado semántico de la palabra completa
Por sub-palabras (estándar actual)	Descompone palabras frecuentes en unidades completas y palabras raras en fragmentos con significado (ej. Byte Pair Encoding)	Permite compartir raíces ("corriendo"/"correr" → "corr"), ahorra memoria y mejora comprensión	Más complejo de implementar

Stopwords y puntuación: las *stopwords* son palabras muy frecuentes (artículos, preposiciones: "el", "de", "y") que actúan como "pegamento" pero no aportan significado temático; se suelen eliminar antes de analizar el significado, junto con signos de puntuación.

```
from nltk.corpus import stopwords
import string

text = "Los programadores están programando programas en Python"
tokens = text.lower().split()
stop_words = set(stopwords.words('spanish'))
filtered = [w for w in tokens if w not in stop_words and w not in
string.punctuation]
# ['programadores', 'están', 'programando', 'programas', 'Python']
```

Stemming vs. Lemming (Lematización):

	Stemming	Lemming (lematización)
Cómo reduce la palabra	Corta el final siguiendo reglas heurísticas	Busca la raíz lingüística (lema) usando un diccionario
Precisión	Puede generar palabras que no existen	Más preciso
Costo	Rápido	Requiere más cómputo
Ejemplo	Corriendo, Corrió, Correrá → "corr"	Corriendo, Corrió, Correrá → "correr" (infinitivo); Mejor, Buenos → "bueno"

```

from nltk.stem import PorterStemmer
ps = PorterStemmer()
stems = [ps.stem(w) for w in filtered] # ['program', 'est', 'program',
'program', 'Python']

from nltk.stem import WordNetLemmatizer
lemmatizer = WordNetLemmatizer()
lemas = [lemmatizer.lemmatize(w) for w in filtered] # ['programador',
'estar', 'programar', 'programa', 'Python']

```

N-gramas: agrupan tokens adyacentes para capturar contexto (frases hechas, conceptos compuestos como "San Francisco"). Unigrama (n=1): palabras sueltas. Bigrama (n=2): pares de palabras consecutivas. Trigrama (n=3): tríos de palabras consecutivas.

```

from nltk.util import ngrams
bigrams = list(ngrams(tokens, 2))
# [('programadores', 'están'), ('están', 'programando'), ...]

```

Representación de texto

Bag of Words (BoW): conjunto de las distintas palabras que aparecen en un documento, asociadas a la cantidad de veces que aparecen (armado de vocabulario + conteo en un vector). Pierde el orden de las palabras.

Ejemplo: D1 = "Todos los estados tienen sus propias leyes"; D2 = "Todos los países tienen sus propias culturas" → vocabulario $V = \{\text{Todos:2, los:2, estados:1, tienen:2, sus:2, propias:2, leyes:1, países:1, culturas:1}\}$, representado como una fila (vector) de conteos por documento.

TF-IDF (Term Frequency - Inverse Document Frequency): métrica de *information retrieval* que pondera qué tan importante es una palabra dentro de un corpus de documentos.

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

Donde **TF** = cantidad de ocurrencias del término t en el documento d ; **IDF** = inversa de la cantidad de documentos donde aparece el término t (un término que aparece en pocos documentos, pero muchas veces en uno, tiene TF-IDF alto — es más "distintivo" de ese documento).

Embeddings: representación vectorial densa de cada token, obtenida con una red neuronal (típicamente arquitecturas Encoder-Decoder). El **Encoder** comprime la representación dispersa (ej. Bag of Words de 1000 documentos) a un vector de pocas dimensiones; una vez entrenado, se descarta el Decoder y se usa solo el Encoder para obtener representaciones de menor dimensionalidad. Esto habilita el **álgebra de vectores**: qué tan parecida/cercana es una palabra a otra en el espacio vectorial (conceptos similares "rotan" de forma similar si se transforma el espacio).

Mean Pooling: para representar una oración/documento completo, se promedian (componente a componente) los embeddings de sus palabras — técnica que preserva la representación espacial y abre la puerta a muchas aplicaciones de NLP y LLMs.

Métricas de similitud entre vectores: **Similitud coseno** (ángulo entre vectores) y **Distancia Euclidiana** (distancia geométrica); son aproximadamente equivalentes si los vectores están normalizados.

Aplicaciones de NLP con embeddings

- **Clusterización de texto:** agrupar documentos por similitud sin conocer su contenido de antemano, reduciendo dimensionalidad (PCA, T-SNE) y clusterizando (HDBSCAN, K-Means). Usos: agrupar/estructurar documentos legales, análisis social (sesgos según medios seguidos).
- **Clasificación de texto:** entrenar un clasificador binario/multiclase sobre los embeddings (ej. identificar noticias falsas, discursos de odio, señales de depresión). Retos: alta cantidad de features (conviene reducir dimensionalidad) y necesidad de datos variados y suficientes.
- **Análisis de sentimiento:** clasificador multiclase sobre etiquetas como Feliz/Triste/Neutral, o binario Positivo/Negativo, o granular (Muy Negativo...Muy Positivo), o por emoción (Alegría, Enojo, Tristeza). Reto principal: **ironía y sarcasmo** — un modelo básico puede malinterpretar "¡Qué maravilla, mi pedido llegó roto!" como positivo por la palabra "maravilla".

Clase 8 — Redes Neuronales Convolucionales (CNNs)

Visión artificial (Computer Vision)

Campo antiguo que busca procesar imágenes y obtener distintos outputs de ellas; con Deep Learning retomó fuerza en los últimos años. Extraer "features" de una imagen a mano es difícil por: variación del punto de vista, deformaciones, oclusiones, variaciones de iluminación, fondos confusos y variaciones intra-clase.

¿Qué es una CNN?

Una **Red Neuronal Convolucional (CNN)** es un tipo de red diseñada específicamente para procesar datos con estructura de grilla (imágenes). Introducidas por **Yann LeCun** en los años 90; usos: clasificación de imágenes, reconocimiento de objetos, detección de objetos, segmentación semántica, e incluso procesamiento de audio.

¿Por qué CNN y no redes densas (fully connected)?

Limitación de las redes densas: al aplicarlas a imágenes, cada píxel se conecta con todas las neuronas de la capa siguiente → cantidad enorme de parámetros, y no capturan bien las relaciones espaciales entre píxeles (bordes, patrones).

Ventajas de las CNN:

- **Convoluciones locales:** exploran solo una pequeña ventana (filtro/kernel) por vez → detectan características locales (bordes, texturas).
- **Reducción de parámetros:** el mismo filtro se aplica a distintas regiones de la imagen (pesos compartidos).
- **Invariancia espacial:** una característica (ej. un borde) puede detectarse en cualquier parte de la imagen — más robusta a traslaciones.

Componentes principales de una CNN

Componente	Qué hace
Capa convolucional	Aplica filtros/kernels (pequeños conjuntos de pesos que se deslizan sobre la imagen) para extraer características como bordes, esquinas o patrones cada vez más complejos en capas más profundas

Componente	Qué hace
Padding	Agrega píxeles extra (generalmente ceros) alrededor de los bordes antes de convolucionar, para preservar información de los bordes, controlar el tamaño de la salida y facilitar apilar múltiples capas sin que la imagen se achique demasiado
Capa de Pooling	Reduce la dimensionalidad manteniendo lo más importante. Max Pooling : toma el valor máximo de cada región. Average Pooling : toma el promedio de cada región
Capa densa (fully connected)	Tras las capas convolucionales y de pooling, procesa las características extraídas para producir la clasificación final

La idea de "conectar parches": las neuronas ocultas solo ven los valores dentro de su parche (ventana deslizante, ej. 3x3 con salto/*stride* 2); por eso capas sucesivas tienen tamaño cada vez más reducido (salvo que se use padding).

Cómo funciona una CNN (paso a paso)

1. **Entrada:** la imagen se convierte en una grilla de píxeles (matriz o tensor).
2. **Convolución (aplicación de filtros):** la capa convolucional aplica múltiples filtros, cada uno detectando un tipo de característica (bordes horizontales, verticales, etc.).
3. **Pooling (reducción de dimensionalidad):** reduce la resolución de las características detectadas, concentrándose en las más importantes y reduciendo el costo computacional.
4. **Clasificación:** las capas densas toman la salida procesada y producen la predicción final (ej. qué dígito representa la imagen).

A medida que las capas convolucionales son más profundas, la red aprende representaciones cada vez más complejas (de bordes simples a patrones/objetos completos).

CNNs en TensorFlow / Keras

```
from tensorflow.keras import layers, Sequential

model = Sequential([
    layers.Conv2D(32, (3, 3), activation="relu", input_shape=(28, 28, 1)),
    # convolución
    layers.MaxPooling2D((2, 2)),
    # pooling
    layers.Conv2D(64, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    # aplana antes de las densas
    layers.Dense(64, activation="relu"),
    layers.Dense(10, activation="softmax")
    # clasificación final
])
```

Capas clave: `Conv2D` (aplica la convolución), `MaxPooling2D` (reduce dimensionalidad), `Flatten` (aplana la salida antes de las capas densas).

Data Augmentation

Proceso de crear nuevas muestras de entrenamiento a partir de las existentes mediante transformaciones (rotación, traslación, zoom, flip horizontal/vertical, escalado, recortes). Ayuda a mejorar la generalización y evitar overfitting, sobre todo con datasets limitados — aunque las capas convolucionales tienen capacidad de ser invariantes a estas transformaciones, hay que **mostrarle ejemplos** al modelo para que aprenda esas invariancias.

Cuidado: aplicar transformaciones que no tienen sentido para el dominio puede perjudicar el modelo (ej. rotar 90° una tomografía computarizada cuando eso nunca ocurriría en la práctica). Se implementa en Keras con `ImageDataGenerator`.

Otras arquitecturas basadas en CNNs

GANs (Generative Adversarial Networks / Redes Generativas Antagónicas, 2014): dos redes que compiten entre sí:

- **Generador (G):** aprende la distribución de probabilidad subyacente a los datos reales y genera nuevas muestras indistinguibles de ellos. Toma como entrada un **vector de ruido aleatorio** (vector latente, la "semilla") y lo transforma en un espacio de alta

dimensionalidad similar al de los datos reales (ej. una imagen), típicamente con capas **deconvolucionales** (convolución transpuesta, que aumentan la dimensionalidad espacial).

- **Discriminador (D):** distingue entre muestras reales del dataset y muestras generadas por G. Suele ser una CNN estándar (convolución + pooling + capas densas), con salida de una sola neurona y activación sigmoide (clasificación binaria real/falsa).

VAEs (Variational Autoencoders / Autoencoders Variacionales, 2013): se basan en inferencia variacional y modelado probabilístico; aprenden una representación latente **probabilística** de los datos de entrada.

- **Codificador (Encoder):** mapea la entrada x a una distribución de probabilidad en el espacio latente (produce media y varianza de esa distribución).
- **Decodificador (Decoder):** toma una muestra z del espacio latente aprendido y la mapea de vuelta al espacio de datos, generando una reconstrucción x' o una nueva muestra similar.

La clave de los VAE es aprender un espacio latente **continuo y bien estructurado**, que permite generar nuevas muestras significativas interpolando y muestreando en ese espacio.

	GANs	VAEs
Componentes	Generador + Discriminador (compiten)	Encoder + Decoder (colaboran)
Enfoque	Adversarial (juego de suma cero)	Probabilístico / inferencia variacional
Espacio latente	No necesariamente estructurado	Continuo y bien estructurado por diseño
Año	2014	2013

Comparación de técnicas de Machine Learning

Modelos de clasificación / regresión

Técnica	Ventajas	Desventajas	¿ Cuándo usarla
Árbol de decisión	Fácil de interpretar y visualizar; no requiere escalado de features; maneja datos numéricos y categóricos	Alta varianza (propenso a overfitting si no se limita la profundidad); fronteras rectangulares, poco suaves	Cuando se necesita interpretabilidad y reglas claras (ej. explicar una decisión de crédito)
SVM	Muy efectivo en datasets chicos/medianos y de alta dimensión; el kernel trick permite fronteras no lineales; robusto con soft margin	Costoso computacionalmente con datasets grandes; sensible a la elección de kernel/hiperparámetros; poco interpretable con kernels no lineales	Datasets chicos/medianos, con frontera de decisión clara y margen amplio entre clases
Random Forest (Bagging)	Reduce variance respecto a un árbol individual; mejor generalización; permite estimar importancia de features; usa out-of-bag como validación gratuita	Menos interpretable que un árbol individual; más costoso en memoria/tiempo que un solo modelo	Cuando un árbol individual overfittea y se necesita robustez sin perder demasiada interpretabilidad
AdaBoost (Boosting)	Reduce bias; convierte modelos débiles en un modelo fuerte; buena performance con pocos ajustes	Sensible a outliers y ruido (los pondera cada vez más); entrenamiento secuencial (más lento, no paralelizable fácilmente)	Cuando los modelos base son consistentemente débiles y se puede iterar (datos relativamente limpios)

Técnica	Ventajas	Desventajas	¿ Cuándo usarla
Voting Classifier	Combina fortalezas de distintos algoritmos; simple de implementar sobre modelos ya entrenados	La mejora depende de que los modelos base sean diversos y no correlacionados	Cuando se dispone de varios modelos con performance similar y errores distintos
Redes neuronales / Deep Learning	Aprende representaciones automáticamente (sin feature engineering manual); escala con más datos; funciona muy bien con datos no estructurados (imagen, audio, texto)	Requiere grandes volúmenes de datos y poder de cómputo (GPU); poco interpretable ("caja negra"); más hiperparámetros para ajustar (learning rate, arquitectura, etc.)	Datos no estructurados o de gran volumen, cuando la interpretabilidad no es crítica y hay cómputo/datos suficientes
CNN	Especializada en datos con estructura de grilla (imágenes); menos parámetros que una red densa equivalente; invariante a traslaciones	Requiere muchos datos etiquetados (o data augmentation); costosa de entrenar sin GPU	Visión por computadora: clasificación/detección de imágenes, video, también audio

Aprendizaje supervisado vs. no supervisado vs. reforzado

	Supervisado	No supervisado	Reforzado
Datos	Etiquetados (labeled)	Sin etiquetar (unlabeled)	Estados + acciones + recompensas
Objetivo	Predecir un valor/clase	Encontrar relaciones/patrones	Aprender la mejor acción a tomar
Dificultad de evaluación	Más fácil (hay ground truth)	Más difícil de evaluar/entender	Depende del diseño de la función de recompensa

	Supervisado	No supervisado	Reforzado
Ejemplos	Clasificación de spam, regresión de precios	Clustering de clientes, PCA	Vehículos autónomos, juegos de estrategia

Bagging vs. Boosting

	Bagging	Boosting
Entrenamiento	Modelos en paralelo , independientes	Modelos en secuencia , cada uno corrige al anterior
Qué reduce	Variance error	Bias error
Muestreo	Subdatasets aleatorios con reemplazo (bootstrap)	Mismo dataset, pero reponderando instancias mal clasificadas
Sensibilidad a outliers	Baja (el promedio diluye el efecto)	Alta (los outliers acumulan peso)
Ejemplo	Random Forest	AdaBoost

Machine Learning "clásico" vs. Deep Learning

	ML clásico (árboles, SVM, regresión)	Deep Learning
Feature engineering	Manual	Automático (aprendido)
Cantidad de datos necesaria	Puede funcionar con datasets chicos/medianos	Necesita grandes volúmenes
Interpretabilidad	Alta (árboles) a media (SVM)	Baja ("caja negra")
Datos no estructurados	Escala mal	Es su fortaleza
Costo computacional	Bajo a medio	Alto (requiere GPU)

Regresión lineal vs. polinomial

Aumentar el grado del polinomio incrementa la complejidad del modelo: permite capturar relaciones más complejas (evita underfitting) pero aumenta el riesgo de overfitting. Se debe validar con curvas de aprendizaje/validación para elegir el grado óptimo.

Consolidado de métricas de evaluación

Clasificación

Métrica	Fórmula	Interpretación práctica	Cuándo priorizarla
Accuracy	$(TN+TP) / (TN+TP+FN+FP)$	% de predicciones totales correctas	Clases balanceadas, sin preferencia entre tipos de error
Precision	$TP / (TP+FP)$	De lo que predije positivo, cuánto era realmente positivo	Cuando el costo de un falso positivo es alto (ej. marcar contenido legítimo como spam)
Recall (Sensitivity)	$TP / (TP+FN)$	De lo que era realmente positivo, cuánto detecté	Cuando el costo de un falso negativo es alto (ej. no detectar una enfermedad o un fraude)
F1-Score	$2 \times (P \times R) / (P+R)$	Media armónica de Precision y Recall; penaliza fuerte si uno de los dos es bajo	Cuando se necesita balancear Precision y Recall a la vez
FPR (False Positive Rate)	$FP / (FP+TN)$	De los negativos reales, cuántos predije (mal) como positivos	Insumo de la curva ROC
TPR (True Positive Rate)	$TP / (TP+FN)$ (= Recall)	De los positivos reales, cuántos detecté	Insumo de la curva ROC
AUC (Área bajo la curva ROC)	Área bajo la curva TPR vs. FPR	Valor entre 0.5 (azar) y 1 (perfecto): qué tan bien separa el modelo las clases, para todos los thresholds a la vez	Comparar clasificadores de forma independiente del threshold elegido

Matriz de confusión (base de todas las métricas anteriores):

	Predicho: No	Predicho: Sí
Real: No	TN	FP (Error tipo I)
Real: Sí	FN (Error tipo II)	TP

Accuracy Paradox: con clases desbalanceadas, un modelo con mayor accuracy puede ser menos útil que uno con menor accuracy (ej. predecir siempre "no" cuando el 90% de los casos son "no"). Por eso, ante clases desbalanceadas conviene mirar Precision/Recall/F1/AUC en vez de (o además de) Accuracy.

Trade-off Precision ↔ Recall: se controla moviendo el **decision threshold** del clasificador; subir el umbral favorece Precision, bajarlo favorece Recall. No existe un threshold óptimo universal: depende del costo relativo de cada tipo de error en el problema.

Regresión

Métrica	Fórmula	Interpretación práctica	Cuándo priorizarla
RMSE	$\sqrt{(1/m) \sum (h(x^i) - y^i)^2)}$	Desvío estándar del error; en las mismas unidades que y	Cuando los errores grandes deben penalizarse fuertemente
MAE	$(1/m) \sum h(x^i) - y^i $	Error absoluto promedio	Cuando se quiere robustez ante outliers/valores extremos

Curvas de diagnóstico (no son una única métrica, pero se leen en el examen igual)

Curva	Eje X	Eje Y	Para qué sirve
Curva ROC	FPR	TPR	Comparar clasificadores variando el threshold; su resumen numérico es el AUC
Curva de aprendizaje	Tamaño del training set	Accuracy (u otra métrica) en train y validación	Decidir si conviene conseguir más datos
Curva de validación / complejidad	Valor de un hiperparámetro	Accuracy (u otra métrica) en train y validación	Detectar en qué zona hay overfitting o underfitting según ese hiperparámetro

Regla general para elegir métrica en un examen

1. ¿Clases balanceadas y sin diferencia de costo entre errores? → **Accuracy**.
2. ¿Clases desbalanceadas? → nunca Accuracy sola; usar **Precision/Recall/F1/AUC**.
3. ¿Un falso positivo es más caro? → **Precision**.
4. ¿Un falso negativo es más caro? → **Recall**.
5. ¿Necesito un solo número que balancee ambos? → **F1**.
6. ¿Quiero comparar modelos sin fijar un threshold? → **AUC**.
7. ¿Es un problema de regresión? → **RMSE** (penaliza errores grandes) o **MAE** (robusto a outliers).

Glosario de términos (inglés / español)

Ciencia de Datos y datos

Término	Definición
Big Data	Conjuntos de datos caracterizados por las 5V: volumen, velocidad, variedad, veracidad y valor
Data lake / Data warehouse	Repositorio de datos crudos sin procesar / repositorio de datos estructurados y procesados para análisis

Término	Definición
Feature / Atributo	Variable o característica medida de una instancia/observación
Instance / Sample / Muestra	Cada observación individual del dataset
Label / Etiqueta	Valor objetivo (clase o número) que se quiere predecir
Feature Engineering	Selección, extracción o creación de atributos para mejorar el modelo
One-Hot Encoding	Codificación de una variable categórica en columnas binarias (una por categoría)
Label Encoding	Codificación de una variable categórica como números enteros
Feature Scaling	Llevar los atributos numéricos a una escala comparable (normalización o estandarización)
Normalization / Normalización (Min-Max)	Escalado de un atributo al rango $[0, 1]$
Standardization / Estandarización	Escalado de un atributo a media 0 y desvío estándar 1
Outlier / Valor atípico	Observación que se aleja significativamente del resto de los datos
Pipeline	Secuencia de pasos de procesamiento de datos/modelo

Machine Learning general

Término	Definición
Training set / Set de entrenamiento	Datos usados para entrenar (ajustar) el modelo
Test set / Set de prueba	Datos separados para evaluar la performance final del modelo
Validation set / Set de validación	Datos usados para ajustar hiperparámetros durante el entrenamiento
Overfitting / Sobreajuste	El modelo aprende el ruido/detalle del training set y generaliza mal a datos nuevos

Término	Definición
Underfitting / Subajuste	El modelo es demasiado simple para capturar los patrones de los datos
Bias (sesgo)	Error por suposiciones incorrectas del modelo
Variance (varianza)	Error por sensibilidad del modelo a fluctuaciones del training set
Hyperparameter / Hiperparámetro	Parámetro configurado antes del entrenamiento (no aprendido de los datos)
Cross-Validation / Validación cruzada	Técnica que divide el training set en k folds para validar el modelo de forma más robusta
Grid Search	Búsqueda exhaustiva de hiperparámetros sobre una grilla de valores
Randomized Search	Búsqueda aleatoria de combinaciones de hiperparámetros
Supervised Learning / Aprendizaje supervisado	Aprendizaje a partir de datos etiquetados
Unsupervised Learning / Aprendizaje no supervisado	Aprendizaje a partir de datos sin etiquetar, busca patrones/estructura
Reinforcement Learning / Aprendizaje reforzado	Aprendizaje mediante recompensas y penalidades por acciones tomadas
Decision Boundary / Frontera de decisión	Superficie que separa las clases en el espacio de características

Clasificación, SVM y ensambles

Término	Definición
Decision Tree / Árbol de decisión	Modelo que divide el espacio de datos mediante reglas jerárquicas
Support Vector Machine (SVM)	Modelo que busca el hiperplano/margen óptimo que separa las clases
Support Vector / Vector de soporte	Instancia ubicada en el borde del margen de un SVM

Término	Definición
Kernel	Función que mapea los datos a un espacio de mayor dimensión para encontrar separación lineal
Hard/Soft Margin	Margen que no permite / sí permite violaciones (instancias dentro de la "calle") en un SVM
Ensemble / Ensamble	Combinación de múltiples modelos para mejorar la predicción
Bagging (Bootstrap Aggregating)	Ensamble que entrena modelos en paralelo sobre subconjuntos aleatorios con reemplazo
Boosting	Ensamble que entrena modelos secuencialmente, cada uno corrigiendo al anterior
Random Forest	Bagging aplicado a árboles de decisión
AdaBoost (Adaptive Boosting)	Algoritmo de boosting que reajusta pesos de instancias mal clasificadas
Out-of-bag	Instancias no incluidas en el subset de un modelo de bagging, usadas como validación
One-vs-All (OvA)	Estrategia multiclase: un clasificador binario por clase
One-vs-One (OvO)	Estrategia multiclase: un clasificador binario por cada par de clases

Deep Learning

Término	Definición
Neural Network / Red neuronal	Modelo compuesto por capas de neuronas interconectadas
Neuron / Neurona	Unidad básica de procesamiento que aplica pesos, bias y una función de activación
Weight / Peso	Parámetro que pondera la influencia de una conexión entre neuronas
Bias (de una neurona)	Parámetro que desplaza la salida de una neurona, independiente de la entrada
Activation Function / Función de activación	Función no lineal aplicada a la salida de una neurona (ReLU, Sigmoid, Softmax)

Término	Definición
ReLU	Función de activación que deja pasar los valores positivos y anula los negativos
Loss Function / Función de pérdida	Función que mide la discrepancia entre predicción y valor real
Backpropagation / Retropropagación	Algoritmo que calcula el gradiente de la pérdida respecto a cada peso para actualizarlos
Gradient Descent / Descenso de gradiente	Algoritmo de optimización que ajusta los pesos en dirección opuesta al gradiente de la pérdida
Stochastic Gradient Descent (SGD)	Descenso de gradiente calculado sobre un dato o mini-batch aleatorio en vez de todo el dataset
Learning Rate / Tasa de aprendizaje	Hiperparámetro que controla el tamaño del paso en cada actualización de pesos
Epoch / Época	Una pasada completa por todo el training set
Batch / Mini-batch	Subconjunto de datos usado en una actualización de pesos

NLP

Término	Definición
Tokenization / Tokenización	Proceso de dividir un texto en unidades mínimas (tokens)
Stopwords	Palabras muy frecuentes sin valor semántico relevante (ej. "el", "de")
Stemming	Reducción heurística de una palabra a su raíz (puede generar palabras inexistentes)
Lemmatization / Lematización	Reducción de una palabra a su lema (forma de diccionario)
N-gram / N-grama	Secuencia de N tokens consecutivos
Bag of Words (BoW)	Representación de un texto como conteo de palabras, sin orden
TF-IDF	Métrica que pondera la importancia de un término en un documento respecto al corpus

Término	Definición
Embedding	Representación vectorial densa de una palabra/token aprendida por una red neuronal
Mean Pooling	Promedio de embeddings de palabras para representar una oración/documento
Cosine Similarity / Similitud coseno	Métrica de similitud entre dos vectores basada en el ángulo entre ellos

Visión por computadora (CNN)

Término	Definición
Convolution / Convolución	Operación que aplica un filtro deslizante sobre la imagen para extraer características
Filter / Kernel	Pequeña matriz de pesos que se desliza sobre la imagen en la convolución
Stride / Salto	Cantidad de píxeles que se desplaza el filtro en cada paso
Padding	Píxeles extra agregados a los bordes de la imagen antes de convolucionar
Pooling	Operación que reduce la dimensionalidad conservando la información más relevante
Max Pooling	Pooling que toma el valor máximo de cada región
Feature Map / Mapa de activación	Resultado de aplicar un filtro convolucional sobre la entrada
Data Augmentation	Generación de nuevas muestras de entrenamiento mediante transformaciones de las existentes
GAN (Generative Adversarial Network)	Arquitectura con un generador y un discriminador que compiten entre sí
VAE (Variational Autoencoder)	Arquitectura Encoder-Decoder que aprende una representación latente probabilística

Evaluación de modelos

Término	Definición
Confusion Matrix / Matriz de confusión	Tabla que cruza clases predichas vs. reales (TP, TN, FP, FN)
True Positive (TP)	Predicción positiva correcta
True Negative (TN)	Predicción negativa correcta
False Positive (FP) / Error tipo I	Predijo positivo siendo negativo
False Negative (FN) / Error tipo II	Predijo negativo siendo positivo
Accuracy / Exactitud	Proporción de predicciones correctas sobre el total
Precision / Precisión	Proporción de positivos predichos que eran realmente positivos
Recall / Exhaustividad (Sensitivity)	Proporción de positivos reales que fueron detectados
F1-Score	Media armónica entre Precision y Recall
ROC Curve / Curva ROC	Gráfico de TPR vs. FPR variando el threshold
AUC (Area Under the Curve)	Área bajo la curva ROC; resume la capacidad discriminativa del modelo
RMSE (Root Mean Square Error)	Raíz del error cuadrático medio, métrica de error en regresión
MAE (Mean Absolute Error)	Error absoluto medio, métrica de error en regresión robusta a outliers

Posibles preguntas de examen

Conceptuales generales

1. **¿Qué es Machine Learning según la definición de Mitchell? Identificá T, E y P en un ejemplo.** — Un programa aprende de la experiencia E respecto a una tarea T si su desempeño (medido por P) mejora con E. Ej.: filtro de spam → T = clasificar emails, E = emails ya etiquetados, P = % de emails correctamente detectados.

2. **¿Cuál es la diferencia entre desarrollo de software tradicional y Machine Learning?** — En el tradicional, el programador escribe las reglas a mano; en ML, el modelo las infiere a partir de datos, generando programas más cortos y adaptables.
3. **¿Qué son las 5V del Big Data?** — Volumen, Velocidad, Variedad, Veracidad, Valor.
4. **¿Cuál es la diferencia entre aprendizaje supervisado, no supervisado y reforzado?** — Supervisado usa datos etiquetados para predecir; no supervisado busca patrones en datos sin etiquetar; reforzado aprende de recompensas/penalidades por acciones.
5. **¿Qué es overfitting y cómo se detecta/corriges?** — El modelo memoriza el training set y generaliza mal; se detecta cuando el training error es bajo pero el generalization error es alto; se corrige simplificando el modelo, regularizando, o consiguiendo más datos.
6. **¿Qué es underfitting y cómo se corrige?** — El modelo es demasiado simple para los datos; se corrige con un modelo más complejo, mejor feature engineering, o reduciendo restricciones del modelo.

Preprocesamiento y features

7. **¿Por qué es necesario el feature scaling? Diferenciá normalización de estandarización.** — Porque atributos en distintas escalas distorsionan algoritmos basados en distancia. Normalización lleva al rango $[0,1]$ (sensible a outliers); estandarización centra en media 0 y desvío 1 (menos sensible a outliers, rango variable).
8. **¿Qué estrategias existen para manejar valores faltantes?** — Descartar mediciones, descartar el atributo, imputar un valor (media/mediana/moda), o predecirlo con un modelo.
9. **¿Cuál es la diferencia entre Label Encoding y One-Hot Encoding? ¿Cuándo usarías cada uno?** — Label Encoding asigna un entero por categoría (riesgo de orden artificial, sirve para ordinales); One-Hot crea una columna binaria por categoría (evita el orden falso, mejor para nominales sin orden).
10. **¿Qué diferencia a un atributo nominal de uno ordinal y de uno numérico? Da un ejemplo de cada uno.**

Clasificación binaria y validación

11. **Describí el pipeline típico de un problema de clasificación binaria.** — Recolección → preprocesamiento → split train/test → entrenamiento → evaluación.
12. **¿Por qué no alcanza con evaluar un modelo solo en el training set?** — Porque no mide generalización; hay que usar un test set (y opcionalmente validation set) no visto durante el entrenamiento.
13. **¿Para qué sirve el validation set si ya existe el test set?** — Para ajustar/seleccionar hiperparámetros sin "contaminar" el test set, que debe usarse una sola vez al final.

14. **Explicá cómo funciona k-fold cross-validation.** — Se divide el training set en k folds; en cada iteración se entrena con k-1 folds y se valida con el restante, repitiendo k veces.
15. **¿Qué diferencia hay entre Grid Search y Randomized Search?** — Grid Search prueba todas las combinaciones de una lista de valores; Randomized Search prueba N combinaciones al azar (más eficiente con espacios grandes).
16. **¿Cómo funcionan las estrategias multiclase One-vs-All y One-vs-One? ¿Cuántos clasificadores requiere cada una para n clases?** — OvA: n clasificadores (uno por clase). OvO: $n \cdot (n-1) / 2$ clasificadores (uno por par).

Evaluación de modelos

17. **Explicá la "Accuracy Paradox" con un ejemplo.** — Con clases desbalanceadas, un modelo que siempre predice la clase mayoritaria puede tener accuracy alta sin ser útil (ej. predecir siempre "no fraude" cuando el 99% no lo es).
18. **Definí Precision y Recall. ¿Cuándo priorizarías cada una?** — Precision = $TP / (TP + FP)$, priorizándola cuando los falsos positivos son costosos. Recall = $TP / (TP + FN)$, priorizándolo cuando los falsos negativos son costosos.
19. **¿Qué es el F1-score y por qué se usa media armónica y no promedio simple?** — Combina Precision y Recall; la media armónica penaliza más cuando uno de los dos valores es bajo, a diferencia del promedio simple.
20. **¿Qué representa la curva ROC y qué mide el AUC?** — ROC grafica TPR vs. FPR variando el threshold; AUC resume en un solo número (0.5 a 1) qué tan bien separa el modelo las clases.
21. **¿Cómo afecta mover el decision threshold a Precision y Recall?** — Subir el threshold sube Precision y baja Recall; bajarlo hace lo opuesto.
22. **Diferenciá RMSE de MAE. ¿Cuál es más sensible a outliers?** — RMSE eleva al cuadrado el error (penaliza más los errores grandes, más sensible a outliers); MAE usa el valor absoluto (más robusto a outliers).

Modelos avanzados

23. **Explicá qué son los vectores de soporte y cómo funciona el margen (calle) en un SVM.**
24. **¿Qué diferencia hay entre Hard Margin y Soft Margin en SVM?** — Hard Margin no permite ninguna violación (exige separabilidad lineal perfecta, sensible a outliers); Soft Margin permite errores controlados por el hiperparámetro de intolerancia (C).
25. **¿Qué es el kernel trick y para qué se usa?** — Proyecta los datos a un espacio de mayor dimensión donde sí son separables linealmente.

26. **Compará Bagging y Boosting: qué tipo de error reduce cada uno y cómo entrenan sus modelos base.** — Bagging entrena en paralelo y reduce variance; Boosting entrena secuencialmente y reduce bias.
27. **¿Qué es Random Forest y por qué generaliza mejor que un árbol de decisión individual?** — Es Bagging de árboles de decisión; al promediar/votar entre muchos árboles entrenados con subsets distintos, reduce la varianza sin aumentar mucho el bias.
28. **¿Qué es el bias-variance tradeoff?** — A mayor complejidad del modelo, típicamente baja el bias pero sube la variance (y viceversa); hay un punto óptimo de complejidad que minimiza el error total.

Deep Learning

29. **¿Por qué son necesarias las funciones de activación no lineales en una red neuronal?** — Sin ellas, la red equivaldría a una combinación lineal de las entradas, sin importar cuántas capas tenga, limitada a resolver problemas lineales.
30. **¿Cuándo usarías Sigmoid vs. Softmax en la capa de salida?** — Sigmoid para clasificación binaria (una probabilidad entre 0 y 1); Softmax para clasificación multiclase (distribución de probabilidad que suma 1).
31. **Explicá los 4 pasos del algoritmo de backpropagation / descenso de gradiente.** — Extracción de datos → forward pass (predicción) → cálculo de la pérdida → ajuste de pesos en la dirección que minimiza la pérdida.
32. **¿Qué efecto tiene un learning rate muy alto vs. muy bajo?** — Muy alto: converge rápido pero puede saltarse el mínimo o ser inestable. Muy bajo: converge de forma precisa pero muy lento, o queda atrapado en mínimos locales.
33. **¿Qué ventaja tiene el descenso de gradiente estocástico (SGD) frente al batch completo?** — Es más rápido por iteración y puede escapar de mínimos locales más fácilmente, aunque el gradiente calculado es más ruidoso.

CNNs

34. **¿Por qué no conviene usar redes densas (fully connected) directamente sobre imágenes?** — Generan demasiados parámetros y no capturan bien las relaciones espaciales entre píxeles.
35. **¿Qué hace una capa convolucional y qué es un kernel/filtro?** — Aplica un pequeño conjunto de pesos (filtro) que se desliza sobre la imagen detectando patrones locales (bordes, texturas).
36. **¿Para qué sirve el padding en una CNN?** — Preservar la información de los bordes, controlar el tamaño de salida y permitir apilar más capas sin que la imagen se reduzca demasiado.

37. **¿Cuál es la diferencia entre Max Pooling y Average Pooling?** — Max Pooling toma el valor máximo de cada región; Average Pooling toma el promedio.
38. **¿Qué es Data Augmentation y por qué ayuda a evitar overfitting?** — Genera nuevas muestras de entrenamiento aplicando transformaciones (rotación, zoom, flip, etc.), mejorando la generalización del modelo ante datasets limitados.
39. **Compará GANs y VAEs: qué componentes tienen y cómo se relacionan entre sí.** — GANs: Generador y Discriminador que compiten (adversarial). VAEs: Encoder y Decoder que colaboran, aprendiendo un espacio latente probabilístico.

NLP

40. **¿Qué diferencia hay entre Stemming y Lematización?** — Stemming corta heurísticamente el final de la palabra (rápido, puede generar palabras inexistentes); Lematización busca la raíz lingüística real usando un diccionario (más preciso, más costoso).
41. **¿Qué mide TF-IDF y por qué es mejor que Bag of Words puro en algunos casos?** — Pondera la frecuencia de un término en un documento por la inversa de en cuántos documentos aparece, destacando términos distintivos en vez de solo frecuentes.
42. **¿Qué es un embedding y qué ventaja tiene sobre Bag of Words?** — Una representación vectorial densa y de baja dimensión aprendida por una red neuronal, que captura relaciones semánticas (palabras similares quedan cerca en el espacio vectorial), a diferencia del vector disperso y sin semántica de BoW.
43. **¿Qué desafío particular presenta el análisis de sentimiento?** — La ironía y el sarcasmo, que pueden hacer que un modelo básico interprete may palabras positivas en un contexto realmente negativo.